

**LAPORAN PRAKTIKUM
PRAKTIKUM DEEP LEARNING**



NAMA : Steven

NIM : 2355301191

KELAS : 3 TI D

DOSEN : Dr. Juni Nurma Sari, S.Kom., M.MT.

**PROGRAM STUDI TEKNIK INFORMATIKA
POLITEKNIK CALTEX RIAU
TA 2025 / 2026**

Klasifikasi dengan Iris dataset menggunakan MLPClassifier

```
[1]: import pandas as pd
      from sklearn.model_selection import train_test_split
      from sklearn.preprocessing import StandardScaler
      from sklearn.neural_network import MLPClassifier
      from sklearn.metrics import classification_report, accuracy_score

      url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv"
```

Melakukan import library dan beberapa fitur dari sklearn. Library pandas yang digunakan untuk mengolah data. Library train_test_split digunakan untuk membagi data menjadi dua bagian yaitu untuk testing dan training. Selain itu terdapat library StandardScaler yang digunakan untuk menyamakan skala fitur yang ada agar model lebih stabil. Library MLPClassifier merupakan model utama algoritma deep learning yang akan digunakan dalam training nanti. Terakhir ada library classification_report & accuracy_score yang digunakan untuk mengavaluasi hasil akhir dan berapa persen akurasi prediksi model.

```
[2]: col_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']
      data = pd.read_csv(url, header=None, names=col_names)
```

Tahap data loading, dimana kita memasukkan data mentah dari internet ke dalam program agar dapat diolah. Selain itu kita menentukan nama kolomnya secara langsung lalu membaca nya dan disimpan pada variabel data.

```
[3]: print(data.head())
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Tahap ini dilakukan untuk menampilkan 5 baris data teratas pada dataset yang akan kita gunakan.

```
[4]: print(data.info())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   sepal_length    150 non-null   float64
 1   sepal_width     150 non-null   float64
 2   petal_length    150 non-null   float64
 3   petal_width     150 non-null   float64
 4   species         150 non-null   object  
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
```

Perintah ini dilakukan untuk membaca informasi singkat mengenai struktur dan karakteristik dari dataframe mengenai jumlah baris data dan berapa kolom beserta tipe data pada setiap kolom.

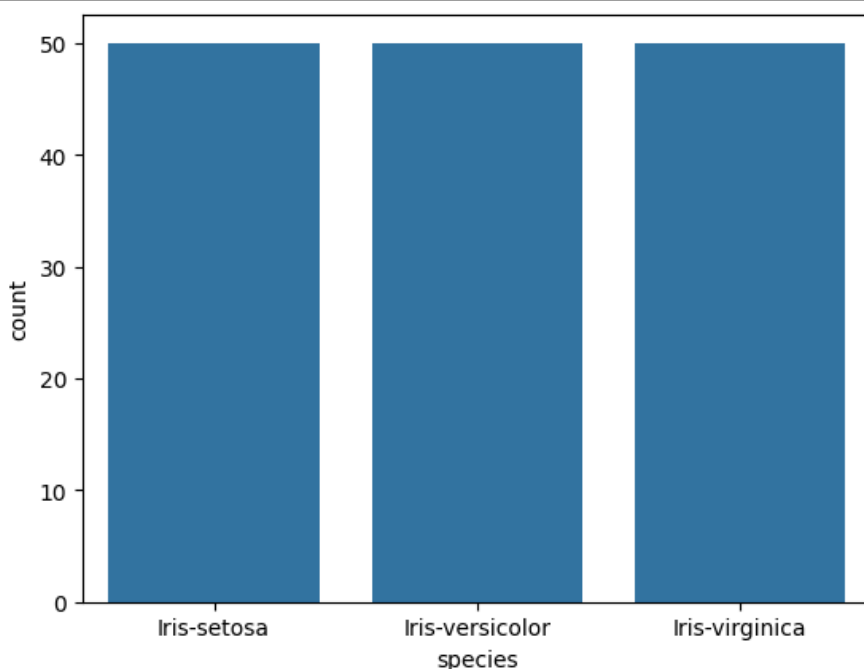
```
[5]: data.isna().sum()

[5]: sepal_length    0
     sepal_width     0
     petal_length    0
     petal_width     0
     species         0
     dtype: int64
```

Perintah ini dilakukan untuk menghitung jumlah total baris data yang null pada setiap kolom data. Dimana fungsi isna() digunakan untuk mencari seluruh baris data di dataframe dan mencari nilai null tersebut dan sum digunakan untuk menghitung nilai yang null tersebut. Dapat diketahui pada dataframe atau dataset kita ini tidak ada baris data yang null.

```
[6]: import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x='species',data=data)
plt.show()
```



Pada tahap ini dilakukan visualisasi dengan menggunakan `sns.countplot()` yang berfungsi dalam menghitung frekuensi kemunculan kategori pada spesies dan menampilkannya dalam bentuk bar atau grafik batang.

```
[7]: data['species'] = data['species'].astype('category').cat.codes
```

Pada tahap ini dilakukan untuk mengubah teks pada kolom species menjadi tipe data kategori dalam bentuk angka misalnya 0,1,2 agar machine dapat memahaminya.

```
[8]: X = data.drop('species', axis=1)
y = data['species']
```

Perintah ini dilakukan untuk memisahkan fitur dan target dalam variabel x dan y dimana x adalah fitur semua kolom kecuali species dan y menyimpan kolom species aja yang berarti label yang ingin kita prediksi

```
[9]: X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2, random_state=42)
```

Pada tahap ini, dilakukan pembagian data untuk testing dan training dengan persentase data untuk testing 0,2 dan training sebesar 0,8 yang dimana testing disimpan untuk melakukan pengujian diakhir.

```
[10]: scaler = StandardScaler()
      X_train = scaler.fit_transform(X_train)
      X_test = scaler.transform(X_test)
```

Pada tahap ini dilakukan scaling dan standarisasi pada data testing dan training karena Neural Network sensitif terhadap rentang angka yang berbeda. Perintah

```
[11]: mlp = MLPClassifier(
      hidden_layer_sizes=(10,10),
      activation='relu',
      solver='adam',
      max_iter=1000)
```

Pada tahap ini dilakukan pembuatan arsitektur model dimana terdapat dua layer tersembunyi dengan 10 neuron masing-masing dengan perintah `hidden_layer_sizes=(10, 10)`. Selanjutnya menggunakan fungsi aktivasi ReLU. Setelah itu penggunaan algoritma optimasi dengan perintah `solver='adam'`.

```
[12]: #Melatih model
      history = mlp.fit(X_train, y_train)
```

Pada tahap ini dilakukan pelatihan model dengan arsitektur yang sudah dirancang sebelumnya dan dimasukan data testing beserta data trainingnya kedalam kemudian hasil dari model disimpan dalam variabel `history`.

```
[13]: y_pred = mlp.predict(X_test)
```

Pada tahap ini melakukan prediksi dengan data testing tersebut dan hasil dari prediksi disimpan dalam variabel `y_pred`.

```
[14]: accuracy = accuracy_score(y_test, y_pred)
      print(f'Akurasi: {accuracy}')

      Akurasi: 0.9666666666666667
```

Pada tahap ini juga dilakukan perhitungan akurasi, dimana akurasi didapatkan berdasarkan hasil yang diprediksi dari `y_pred` dan dicocokkan dengan label fitur hasil dari `y_test` ibaratnya kunci jawaban dan mendapatkan akurasi di angka 0.96.

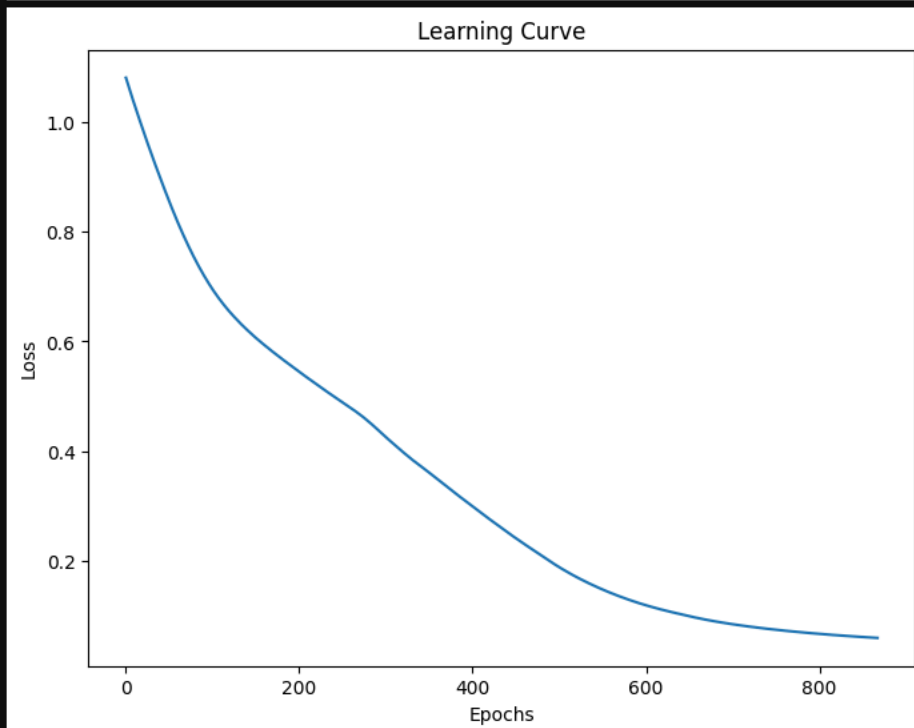
```
[15]: print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	0.89	0.94	9
2	0.92	1.00	0.96	11
accuracy			0.97	30
macro avg	0.97	0.96	0.97	30
weighted avg	0.97	0.97	0.97	30

Pada tahap ini mencetak hasil prediksi yang lebih detail dari akurasi aja. Dimana dapat mengetahui kategori 0,1,2 masing-masing akurasi.

```
[16]: import seaborn as sns
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(8,6))
plt.plot(mlp.loss_curve_)
plt.title('Learning Curve')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.show()
```



Pada tahap ini, melakukan visualisasi grafik learning curve berdasarkan nilai loss selama proses pelatihan. Berdasarkan grafik semakin banyak epoch atau pelatihan maka semakin rendah tingkat lossnya yang berarti model ini termasuk sehat dan bagus.

Membuat klasifikasi dengan Iris dataset menggunakan Sequential

```
[17]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input

# Load Dataset
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'class' ]
dataset = pd.read_csv(url, names=names)
```

Melakukan import library dan beberapa fitur dari sklearn. Library pandas yang digunakan untuk mengolah data. Library train_test_split digunakan untuk membagi data menjadi dua bagian yaitu untuk testing dan training. Selain itu terdapat library StandardScaler yang digunakan untuk menyamakan skala fitur yang ada agar model lebih stabil. Library tensorflow yang merupakan framework deep learning buatan dari google dimana penggunaan model Sequential yang memiliki lapisan saraf yang disusun secara berurutan dari input ke output. Setelah itu ada Dense dan input yang merupakan lapisan saraf yang memiliki neuron terhubung dan Input digunakan untuk mendefinisikan bentuk data awal yang akan masuk ke model.

Selanjutnya mengambil dataset dari internet dan di load ke dalam program dengan pemberian nama kolom pada data dan disimpan dalam sebuah dataframe.

```
[18]: dataset.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   sepal-length    150 non-null   float64
 1   sepal-width     150 non-null   float64
 2   petal-length    150 non-null   float64
 3   petal-width     150 non-null   float64
 4   class           150 non-null   object  
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

Perintah ini dilakukan untuk membaca informasi singkat mengenai struktur dan karakteristik dari dataframe mengenai jumlah baris data dan berapa kolom beserta tipe data pada setiap kolom.

```
[19]: dataset.head()
```

	sepal-length	sepal-width	petal-length	petal-width	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Pada perintah ini, dilakukan untuk menampilkan 5 data teratas dari dataframe atau dataset yang telah di load dari internet.

```
[20]: X = dataset.iloc[:, :-1].values
      y = dataset.iloc[:, -1].values
```

Perintah .iloc disini digunakan untuk memisahkan data untuk membagi dataset menjadi 2 komponen utama. Dimana[:, :-1] berfungsi untuk mengambil semua baris dan semua kolom kecuali kolom terakhir sedangkan fitur[:, -1] digunakan untuk mengambil semua baris tetapi hanya kolom paling terakhir aja.

```
[21]: from sklearn.preprocessing import LabelEncoder
      encoder = LabelEncoder()
      y= encoder.fit_transform(y)
```

Pada tahapan ini melakukan encoding label dengan LabelEncoder yang berfungsi mengonversi label seperti iris-setosa dan iris-versicolor menjadi format angka (0,1,2)

```
[22]: X_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Melakukan pemisahan data menjadi data testing dan data training pada masing-masing variabel x dan y yang mewakili fitur dan label yang ingin di prediksi. Dimana persentase data test yang digunakan yaitu 0,2 dan training 0,8.

```
[23]: scaler = StandardScaler()
      X_train = scaler.fit_transform(X_train)
      X_test = scaler.transform(X_test)
```

Pada tahap ini, melakukan standarisasi pada data set fitur variabel x yang berfungsi agar model dapat memahaminya dan tidak terjadi selisih yang terlalu besar yang menyebabkan nilai dengan angka tinggi dianggap menjadi lebih penting.


```
[24]: model = Sequential()
      model.add(Input(shape=(4,))),
      model.add(Dense(32, activation='relu'))
      model.add(Dense(16, activation='softmax'))
```

Pada tahap ini, dilakukan pembuatan arsitektur model, dimana `Input(shape=(4,))` menandakan model ini menerima 4 input, `Dense 32` dengan `activation relu` yang berarti hidden layer pertama dengan 32 neuron dan fungsi ReLU digunakan agar model bisa mempelajari pola non-linear yang kompleks. Selanjutnya 16 dense berarti 16 neuron dengan softmax yang merupakan layer output yang mengubah hasil prediksi menjadi nilai probabilitas seperti 1.0 atau 100%

```
[25]: model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics = ['accuracy'])
```

Pada tahap ini melakukan kompilasi model agar model dikonfigurasi menggunakan peraturan tertentu. Dimana `loss='sparse_categorical_crossentropy'` berfungsi untuk menghitung seberapa jauh kesalahan prediksi model. `optimizer='adam'` berfungsi untuk memperbarui bobot neuron agar kesalahan semakin kecil. Selanjutnya `metrics=['accuracy']` yang berfungsi dalam menampilkan persentase akurasi selama proses training.

```
[26]: history= model.fit(X_train, y_train, validation_split=0.15, batch_size=10, epochs=15, verbose=1)

Epoch 1/15
11/11 ————— 0s 11ms/step - accuracy: 0.1471 - loss: 2.5182 - val_accuracy: 0.5000 - val_loss: 2.4292
Epoch 2/15
11/11 ————— 0s 4ms/step - accuracy: 0.5098 - loss: 2.3455 - val_accuracy: 0.6667 - val_loss: 2.2818
Epoch 3/15
11/11 ————— 0s 4ms/step - accuracy: 0.6176 - loss: 2.1769 - val_accuracy: 0.6667 - val_loss: 2.1359
Epoch 4/15
11/11 ————— 0s 4ms/step - accuracy: 0.6471 - loss: 2.0107 - val_accuracy: 0.6667 - val_loss: 1.9948
Epoch 5/15
11/11 ————— 0s 4ms/step - accuracy: 0.6471 - loss: 1.8506 - val_accuracy: 0.6667 - val_loss: 1.8552
Epoch 6/15
11/11 ————— 0s 4ms/step - accuracy: 0.6569 - loss: 1.6960 - val_accuracy: 0.6667 - val_loss: 1.7188
Epoch 7/15
11/11 ————— 0s 4ms/step - accuracy: 0.6569 - loss: 1.5456 - val_accuracy: 0.6667 - val_loss: 1.5854
Epoch 8/15
11/11 ————— 0s 5ms/step - accuracy: 0.6863 - loss: 1.4058 - val_accuracy: 0.6667 - val_loss: 1.4585
Epoch 9/15
11/11 ————— 0s 5ms/step - accuracy: 0.7353 - loss: 1.2773 - val_accuracy: 0.6667 - val_loss: 1.3392
Epoch 10/15
11/11 ————— 0s 4ms/step - accuracy: 0.7647 - loss: 1.1605 - val_accuracy: 0.7222 - val_loss: 1.2267
Epoch 11/15
11/11 ————— 0s 4ms/step - accuracy: 0.8137 - loss: 1.0565 - val_accuracy: 0.7778 - val_loss: 1.1251
Epoch 12/15
11/11 ————— 0s 4ms/step - accuracy: 0.8137 - loss: 0.9644 - val_accuracy: 0.7778 - val_loss: 1.0360
Epoch 13/15
11/11 ————— 0s 4ms/step - accuracy: 0.8235 - loss: 0.8834 - val_accuracy: 0.7222 - val_loss: 0.9573
Epoch 14/15
11/11 ————— 0s 4ms/step - accuracy: 0.8235 - loss: 0.8131 - val_accuracy: 0.7222 - val_loss: 0.8864
Epoch 15/15
11/11 ————— 0s 4ms/step - accuracy: 0.8235 - loss: 0.7498 - val_accuracy: 0.7222 - val_loss: 0.8237
```

Pada tahap ini melakukan Training model dan menganalisis parameter training pada setiap epochs ketika training.

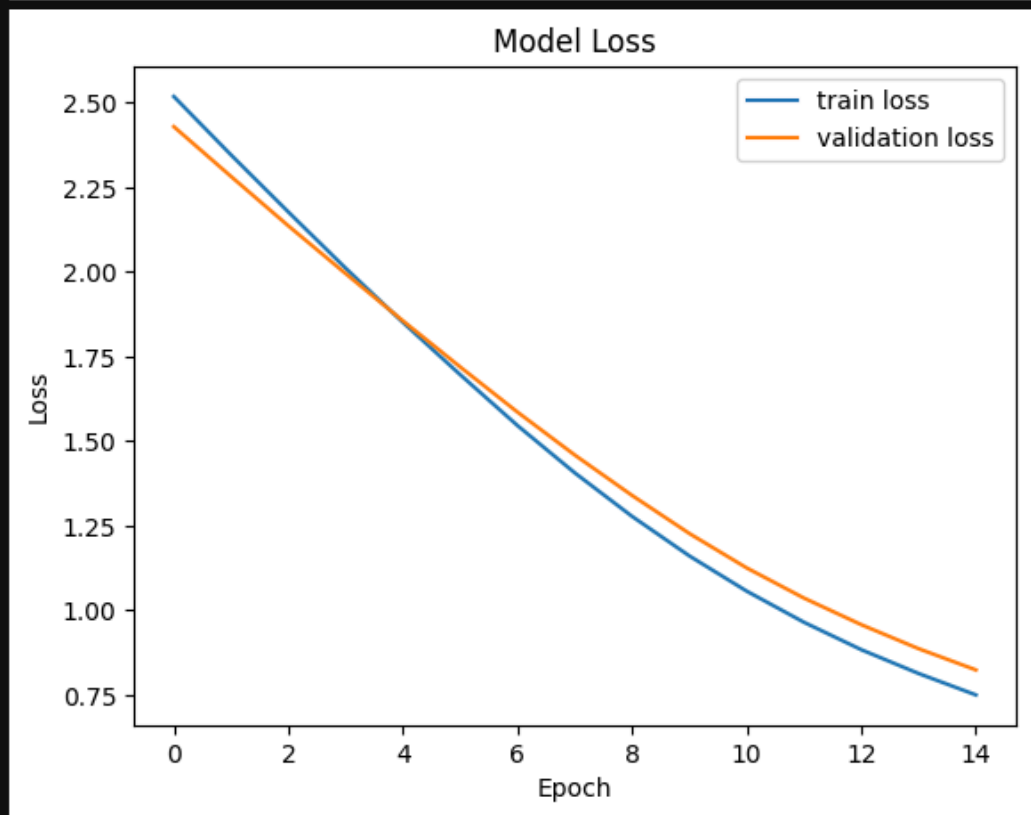
```
[27]: _, accuracy = model.evaluate(X_test, y_test)
      print('Accuracy: %.2f' % (accuracy*100))

      1/1 ————— 0s 21ms/step - accuracy: 0.5667 - loss: 1.2107
      Accuracy: 56.67
```

Pada tahap ini, melakukan evaluasi pada model dengan memprediksi dengan data test dan mencocokkannya dengan hasil `y_test` setelah itu mendapatkan sejumlah akurasi yang benar dari hasil tersebut bahwa akurasinya masih rendah dan angka lossnya sangat tinggi.

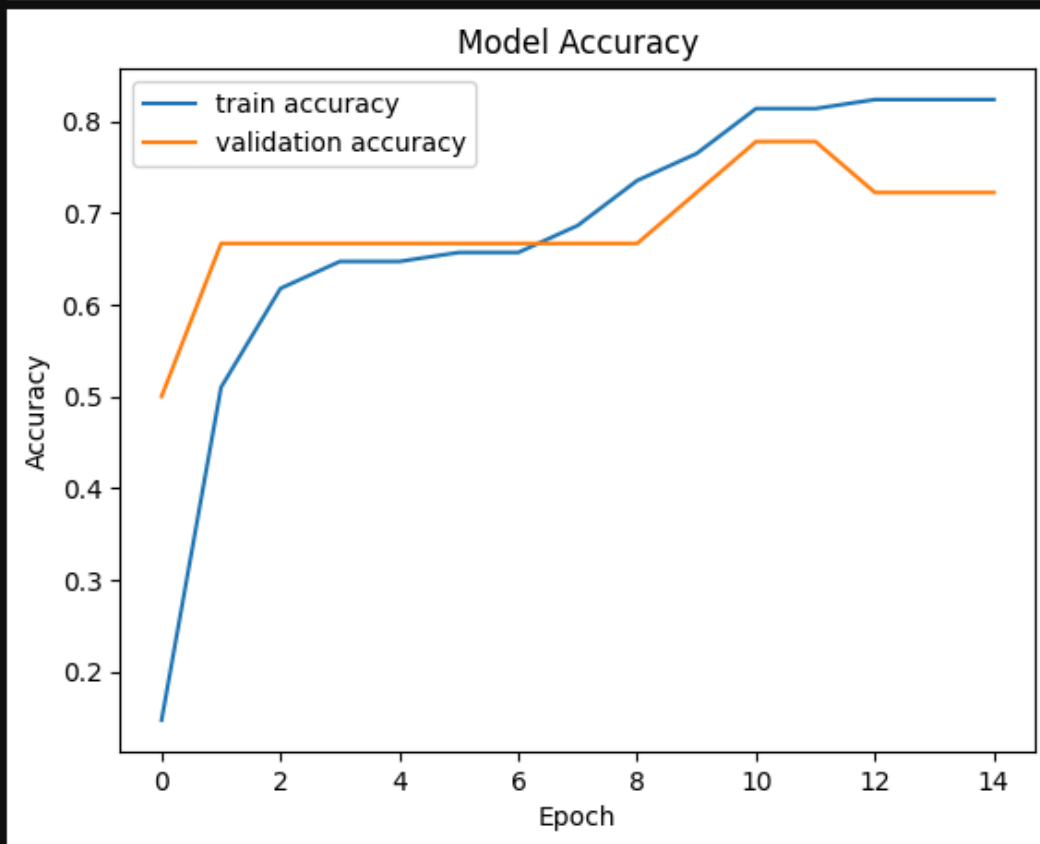
```
[28]: import seaborn as sns
      import matplotlib.pyplot as plt

      plt.plot(history.history['loss'], label='train loss')
      plt.plot(history.history['val_loss'], label='validation loss')
      plt.title('Model Loss')
      plt.ylabel('Loss')
      plt.xlabel('Epoch')
      plt.legend()
      plt.show()
```



Pada tahap ini melakukan visualisasi untuk mengetahui model lossnya dimana dari awal loss tinggi tetapi setiap latihan epoch lossnya turun yang berarti akurasi bisa lebih tinggi lagi jika pelatihan modelnya tetap dilanjutkan.

```
[29]: plt.plot(history.history['accuracy'], label='train accuracy')
plt.plot(history.history['val_accuracy'], label='validation accuracy')
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend()
plt.show()
```



Pada tahapan ini melakukan visualisasi untuk akurasi model, dapat dilihat pada grafik ini dimana pada setiap training berjalan semakin meningkat akurasinya hingga 0.8.

Membuat klasifikasi dengan citrus dataset menggunakan sequential

```
[30]: import pandas as pd
      from sklearn.model_selection import train_test_split
      from sklearn.preprocessing import StandardScaler
      from tensorflow.keras.models import Sequential
      from tensorflow.keras.layers import Dense
      from sklearn.metrics import classification_report, accuracy_score

      data = pd.read_csv('citrus - citrus - citrus - citrus.csv')
```

Melakukan import library dan beberapa fitur dari sklearn. Library pandas yang digunakan untuk mengolah data. Library train_test_split digunakan untuk membagi data menjadi dua bagian yaitu untuk testing dan training. Selain itu terdapat library StandardScaler yang digunakan untuk menyamakan skala fitur yang ada agar model lebih stabil. Library tensorflow yang merupakan framework deep learning buatan dari google dimana penggunaan model Sequential yang memiliki lapisan saraf yang disusun secara berurutan dari input ke output. Setelah itu ada Dense dan input yang merupakan lapisan saraf yang memiliki neuron terhubung dan Input digunakan untuk mendefinisikan bentuk data awal yang akan masuk ke model. Selanjutnya membaca dataset yang di load ke dalam program dengan bentuk csv.

```
[31]: data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype  
---  -
 0   name        10000 non-null  object  
 1   diameter    10000 non-null  float64  
 2   weight      10000 non-null  float64  
 3   red         10000 non-null  int64  
 4   green       10000 non-null  int64  
 5   blue        10000 non-null  int64  
dtypes: float64(2), int64(3), object(1)
memory usage: 468.9+ KB
```

Perintah ini dilakukan untuk membaca informasi singkat mengenai struktur dan karakteristik dari dataframe mengenai jumlah baris data dan berapa kolom beserta tipe data pada setiap kolom.

```
[32]: data.head()
```

```
[32]:
```

	name	diameter	weight	red	green	blue
0	orange	2.96	86.76	172	85	2
1	orange	3.91	88.05	166	78	3
2	orange	4.42	95.17	156	81	2
3	orange	4.47	95.60	163	81	4
4	orange	4.48	95.76	161	72	9

Pada perintah ini, dilakukan untuk menampilkan 5 data teratas dari dataframe atau dataset yang telah dibaca.

```
[33]: X= data.drop('name', axis=1)
      y= pd.get_dummies(data['name'])
```

Perintah ini dilakukan untuk memisahkan fitur dan target dalam variabel x dan y dimana x adalah fitur semua kolom kecuali name dan y menyimpan kolom name aja yang berarti label yang ingin kita prediksi.

```
[34]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Melakukan pemisahan data menjadi data testing dan data training pada masing-masing variabel x dan y yang mewakili fitur dan label yang ingin di prediksi. Dimana persentase data test yang digunakan yaitu 0,2 dan training 0,8.

```
[35]: scaler = StandardScaler()
      X_train = scaler.fit_transform(X_train)
      X_test = scaler.transform(X_test)
```

Pada tahap ini, melakukan standarisasi pada data set fitur variabel x yang berfungsi agar model dapat memahaminya dan tidak terjadi selisih yang terlalu besar yang menyebabkan nilai dengan angka tinggi dianggap menjadi lebih penting.

```
[36]: model = Sequential()
      model.add(Dense(16, activation='relu', input_shape=(X_train.shape[1],)))
      model.add(Dense(9, activation='relu'))
      model.add(Dense(y.shape[1], activation='softmax'))

C:\Users\225-PC27\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\layers\core\dense.py:95: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Pada tahap ini, dilakukan pembuatan arsitektur model, dimana `Input(shape=(4,))` menandakan model ini menerima 4 input, `Dense 16` dengan activation `relu` yang berarti hidden layer pertama dengan 32 neuron dan fungsi ReLU digunakan agar model bisa mempelajari pola non-linear yang kompleks. Selain itu terdapat layer kedua dengan neuron 9 dan activation `relu`. Selanjutnya aktivasi `softmax` yang merupakan layer output yang mengubah hasil prediksi menjadi nilai probabilitas seperti 1.0 atau 100%

```
[37]: model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Pada tahap ini melakukan kompilasi model agar model dikonfigurasi menggunakan peraturan tertentu. Dimana optimizer 'adam' berfungsi dalam memperbarui bobot dan bias disetiap neuron berdasarkan tingkat kesalahan yang ditemukan. `loss='categorical_crossentropy'` berfungsi untuk menghitung seberapa jauh kesalahan prediksi model dan hanya bekerja jika label sudah berbentuk One Hot Encoding seperti `[1,0,0]`. Selanjutnya `metrics=['accuracy']` yang berfungsi dalam menampilkan persentase akurasi selama proses training.

```
[38]: model.fit(X_train, y_train, epochs=50, batch_size=16, validation_split=0.2, verbose=1)

Epoch 1/50
400/400 — 1s 1ms/step - accuracy: 0.8873 - loss: 0.3162 - val_accuracy: 0.9237 - val_loss: 0.1857
Epoch 2/50
400/400 — 0s 824us/step - accuracy: 0.9264 - loss: 0.1823 - val_accuracy: 0.9237 - val_loss: 0.1830
Epoch 3/50
400/400 — 0s 769us/step - accuracy: 0.9278 - loss: 0.1792 - val_accuracy: 0.9237 - val_loss: 0.1818
Epoch 4/50
400/400 — 0s 791us/step - accuracy: 0.9267 - loss: 0.1780 - val_accuracy: 0.9275 - val_loss: 0.1833
Epoch 5/50
400/400 — 0s 819us/step - accuracy: 0.9275 - loss: 0.1774 - val_accuracy: 0.9269 - val_loss: 0.1799
Epoch 6/50
400/400 — 0s 821us/step - accuracy: 0.9270 - loss: 0.1768 - val_accuracy: 0.9281 - val_loss: 0.1803

Epoch 49/50
400/400 — 0s 757us/step - accuracy: 0.9786 - loss: 0.0620 - val_accuracy: 0.9750 - val_loss: 0.0740
Epoch 50/50
400/400 — 0s 772us/step - accuracy: 0.9791 - loss: 0.0620 - val_accuracy: 0.9756 - val_loss: 0.0744
[38]: <keras.src.callbacks.history.History at 0x1a201cedde0>
```

Pada tahap ini model melakukan training dan belajar sebanyak 50 epochs. Selain itu tujuan `batch_size=16` adalah membagi data menjadi kelompok kecil 16 sampel per langkah. Terakhir mendapat akurasi akhir pada epoch yaitu sebesar 0.9791.

Web

```
# Reading the cleaned numeric titanic survival data
import pandas as pd
import numpy as np

# To remove the scientific notation from numpy arrays
np.set_printoptions(suppress=True)

TitanicSurvivalDataNumeric=pd.read_pickle('TitanicSurvivalDataNumeric.pkl')
TitanicSurvivalDataNumeric.head()
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked_C	Embarked_Q	Embarked_S
0	0	3	1	22.0	1	0	7.2500	0	0	1
1	1	1	0	38.0	1	0	71.2833	1	0	0
2	1	3	0	26.0	0	0	7.9250	0	0	1
3	1	1	0	35.0	1	0	53.1000	0	0	1
4	0	3	1	35.0	0	0	8.0500	0	0	1

Pada tahap ini melakukan import library yang akan digunakan. Setelah itu memuat data dari file pickle yang memiliki kecepatan jauh lebih cepat dibaca dari pada file CSV. Setelah itu menampilkan isi dari data yang telah dibaca.

```
# Separate Target Variable and Predictor Variables
TargetVariable=['Survived']
Predictors=['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare',
            'Embarked_C', 'Embarked_Q', 'Embarked_S']

X=TitanicSurvivalDataNumeric[Predictors].values
y=TitanicSurvivalDataNumeric[TargetVariable].values

### Sandardization of data ###
### We does not standardize the Target variable for classification
from sklearn.preprocessing import StandardScaler
PredictorScaler=StandardScaler()
```

Pada tahap ini dilakukan pemisahan data fitur dan label atau target menjadi x dan y. Dimana targetnya adalah Survived. Setelah itu melakukan standarisasi yang bertujuan untuk menyamakan skala data yang disimpan dalam objek PredictorScaler.

```

# Storing the fit object for later reference
PredictorScalerFit=PredictorScaler.fit(X)

# Generating the standardized values of X and y
X=PredictorScalerFit.transform(X)

# Split the data into training and testing set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Quick sanity check with the shapes of Training and Testing datasets
print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)

(623, 9)
(623, 1)
(268, 9)
(268, 1)

```

Pada tahap ini melakukan proses scaling secara langsung pada data fitur yaitu variabel x dan disimpan hasil standarisasi pada variabel PredictorScalerFit. Selanjutnya membagi data untuk testing dan training pada variabel x dan y kemudian mengecek struktur data pada variabel yang tersimpan di data x dan y yang digunakan untuk training dan testing.

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input

classifier = Sequential()
# Defining the Input layer and FIRST hidden layer, both are same!
# relu means Rectifier Linear unit function
classifier.add(Dense(units=10, input_dim=9, kernel_initializer='uniform', activation='relu'))

# Defining the SECOND hidden layer, here we have not defined input because it is
# second layer and it will get input as the output of first hidden layer
classifier.add(Dense(units=6, kernel_initializer='uniform', activation='relu'))

# Defining the Output layer
# sigmoid means sigmoid activation function
# for Multiclass classification the activation = 'softmax'
# And output_dim will be equal to the number of factor levels
classifier.add(Dense(units=1, kernel_initializer='uniform', activation='sigmoid'))

# Optimizer== the algorithm of SGD to keep updating weights
# loss== the loss function to measure the accuracy
# metrics== the way we will compare the accuracy after each step of SGD
classifier.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# fitting the Neural Network on the training data
survivalANN_Model=classifier.fit(X_train,y_train, batch_size=10 , epochs=10, verbose=1)

# fitting the Neural Network on the training data
survivalANN_Model=classifier.fit(X_train,y_train, batch_size=10 , epochs=10, verbose=1)

```



```

Epoch 1/10
C:\Users\225-PC27\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\layers\core\dense.py:95: UserWarning: Do
ot pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object
the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
63/63 ————— 0s 862us/step - accuracy: 0.6228 - loss: 0.6902
Epoch 2/10
63/63 ————— 0s 752us/step - accuracy: 0.6292 - loss: 0.6756
Epoch 3/10
63/63 ————— 0s 679us/step - accuracy: 0.7303 - loss: 0.6107
Epoch 4/10
63/63 ————— 0s 673us/step - accuracy: 0.7994 - loss: 0.5326
Epoch 5/10
63/63 ————— 0s 893us/step - accuracy: 0.8106 - loss: 0.4947
Epoch 6/10
63/63 ————— 0s 831us/step - accuracy: 0.8218 - loss: 0.4756
Epoch 7/10
63/63 ————— 0s 681us/step - accuracy: 0.8186 - loss: 0.4638
Epoch 8/10
63/63 ————— 0s 660us/step - accuracy: 0.8154 - loss: 0.4579
Epoch 9/10
63/63 ————— 0s 691us/step - accuracy: 0.8170 - loss: 0.4509
Epoch 10/10
63/63 ————— 0s 672us/step - accuracy: 0.8138 - loss: 0.4465
Epoch 1/10
63/63 ————— 0s 662us/step - accuracy: 0.8122 - loss: 0.4429
Epoch 2/10
63/63 ————— 0s 660us/step - accuracy: 0.8154 - loss: 0.4401
Epoch 3/10
63/63 ————— 0s 694us/step - accuracy: 0.8170 - loss: 0.4371
Epoch 4/10
63/63 ————— 0s 685us/step - accuracy: 0.8138 - loss: 0.4349
Epoch 5/10
63/63 ————— 0s 691us/step - accuracy: 0.8218 - loss: 0.4334
Epoch 6/10
63/63 ————— 0s 714us/step - accuracy: 0.8202 - loss: 0.4311
Epoch 7/10
63/63 ————— 0s 661us/step - accuracy: 0.8202 - loss: 0.4291
Epoch 8/10
63/63 ————— 0s 649us/step - accuracy: 0.8170 - loss: 0.4285
Epoch 9/10
63/63 ————— 0s 651us/step - accuracy: 0.8154 - loss: 0.4276
Epoch 10/10
63/63 ————— 0s 661us/step - accuracy: 0.8186 - loss: 0.4260

```

Pada tahap ini dilakukan pembangunan arsitektur model dimana lapisan pertama berisi 10 neuron dan memiliki inputan untuk 9 kolom fitur, setelah itu lapisan kedua memiliki 6 neuron dan lapisan output memiliki satu unit dengan aktivasi sigmoid yang berfungsi untuk binary classification

```

# Defining a function for finding best hyperparameters
def FunctionFindBestParams(X_train, y_train):

    # Defining the list of hyper parameters to try
    TrialNumber=0
    batch_size_list=[5, 10, 15, 20]
    epoch_list=[5, 10, 50, 100]

    import pandas as pd
    SearchResultsData=pd.DataFrame(columns=['TrialNumber', 'Parameters', 'Accuracy'])

    for batch_size_trial in batch_size_list:
        for epochs_trial in epoch_list:
            TrialNumber+=1

            # Creating the classifier ANN model
            classifier = Sequential()
            classifier.add(Dense(units=10, input_dim=9, kernel_initializer='uniform', activation='relu'))
            classifier.add(Dense(units=6, kernel_initializer='uniform', activation='relu'))
            classifier.add(Dense(units=1, kernel_initializer='uniform', activation='sigmoid'))
            classifier.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

            survivalANN_Model=classifier.fit(X_train,y_train, batch_size=batch_size_trial , epochs=epochs_trial, verbose=0)
            # Fetching the accuracy of the training
            Accuracy = survivalANN_Model.history['accuracy'][-1]

            # printing the results of the current iteration
            print(TrialNumber, 'Parameters:', 'batch_size:', batch_size_trial, '-', 'epochs:', epochs_trial, 'Accuracy:', Accuracy)

            SearchResultsData.loc[len(SearchResultsData)] = [
                TrialNumber,
                f'batch_size{batch_size_trial}-epoch{epochs_trial}',
                Accuracy
            ]

    return(SearchResultsData)

#####

# Calling the function
ResultsData=FunctionFindBestParams(X_train, y_train)

```

Pada tahap ini melakukan hyperparameter tuning dimana model di setel dengan peraturan yang tidak dipelajari oleh Algoritmanya sendiri seperti epochs dan batchsize. Selanjutnya pembuatan FunctionFindBestParams yang berfungsi untuk mengambil satu nilai dari batch_size_list. Selanjutnya program mencoba semua nilai di epoch_list dari 5,10,50,100 satu per satu.

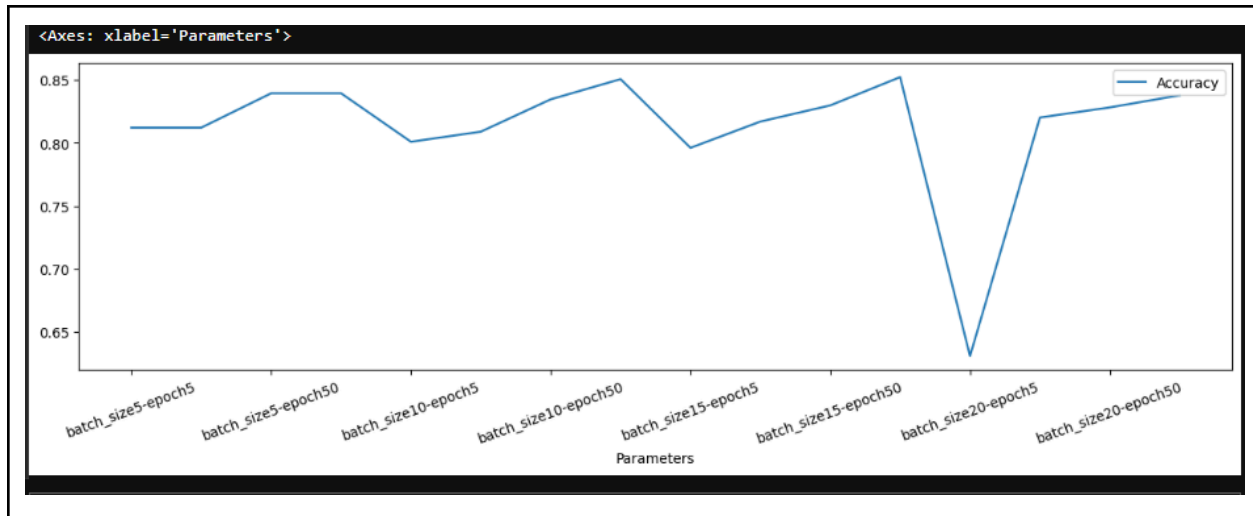
```

[5]: # Printing the best parameter
print(ResultsData.sort_values(by='Accuracy', ascending=False).head(1))

# Visualizing the results
%matplotlib inline
ResultsData.plot(x='Parameters', y='Accuracy', figsize=(15,4), kind='line', rot=20)

```

TrialNumber	Parameters	Accuracy
11	12 batch_size15-epoch100	0.852327



Pada tahap ini melakukan evaluasi hasil dari eksperimen dan memilih model yang terbaik. Dapat di ketahui disini bahwa model saya membutuhkan setidaknya 50-100 epoch dengan batchsize 10-15 untuk mencapai akurasi diatas 80%.

```
[6]: # Training the model with best hyperparamters
      classifier.fit(X_train,y_train, batch_size=5 , epochs=100, verbose=1)

Epoch 1/100
125/125 ————— 0s 589us/step - accuracy: 0.8154 - loss: 0.4269
Epoch 2/100
125/125 ————— 0s 564us/step - accuracy: 0.8186 - loss: 0.4264
Epoch 3/100
125/125 ————— 0s 575us/step - accuracy: 0.8170 - loss: 0.4232
Epoch 4/100
125/125 ————— 0s 571us/step - accuracy: 0.8218 - loss: 0.4205
Epoch 5/100
125/125 ————— 0s 577us/step - accuracy: 0.8250 - loss: 0.4206
Epoch 6/100
125/125 ————— 0s 587us/step - accuracy: 0.8234 - loss: 0.4213
Epoch 7/100
125/125 ————— 0s 575us/step - accuracy: 0.8218 - loss: 0.4198
Epoch 8/100
125/125 ————— 0s 578us/step - accuracy: 0.8186 - loss: 0.4190
Epoch 9/100
125/125 ————— 0s 587us/step - accuracy: 0.8218 - loss: 0.4196
Epoch 10/100
125/125 ————— 0s 579us/step - accuracy: 0.8202 - loss: 0.4181
Epoch 11/100
125/125 ————— 0s 592us/step - accuracy: 0.8234 - loss: 0.4179
Epoch 12/100
125/125 ————— 0s 580us/step - accuracy: 0.8218 - loss: 0.4164
Epoch 13/100
```

Pada tahap ini melakukan final training yang menggunakan parameter terbaik yaitu dengan epochs 100 dan batch 5 dalam training.

```
[7]: # Predictions on testing data
Predictions=classifier.predict(X_test)

# Scaling the test data back to original scale
Test_Data=PredictorScalerFit.inverse_transform(X_test)

# Generating a data frame for analyzing the test data
TestingData=pd.DataFrame(data=Test_Data, columns=Predictors)
TestingData['Survival']=y_test
TestingData['PredictedSurvivalProb']=Predictions

# Defining the probability threshold
def probThreshold(inpProb):
    if inpProb > 0.5:
        return(1)
    else:
        return(0)

# Generating predictions on the testing data by applying probability threshold
TestingData['PredictedSurvival']=TestingData['PredictedSurvivalProb'].apply(probThreshold)
print(TestingData.head())

#####
from sklearn import metrics
print('\n##### Testing Accuracy Results #####')
print(metrics.classification_report(TestingData['Survival'], TestingData['PredictedSurvival']))
print(metrics.confusion_matrix(TestingData['Survival'], TestingData['PredictedSurvival']))
```

```
9/9 0s 3ms/step
```

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked_C	Embarked_Q	\
0	3.0	1.0	23.0	1.0	1.0	15.2458	1.0	0.0	
1	2.0	1.0	31.0	0.0	0.0	10.5000	0.0	0.0	
2	3.0	1.0	20.0	0.0	0.0	7.9250	0.0	0.0	
3	2.0	0.0	6.0	0.0	1.0	33.0000	0.0	0.0	
4	3.0	0.0	14.0	1.0	0.0	11.2417	1.0	0.0	

	Embarked_S	Survival	PredictedSurvivalProb	PredictedSurvival
0	0.0	1	0.304650	0
1	1.0	0	0.118297	0
2	1.0	0	0.133372	0
3	1.0	1	0.943597	1
4	0.0	1	0.505826	1


```
##### Testing Accuracy Results #####
```

	precision	recall	f1-score	support
0	0.78	0.91	0.84	157
1	0.84	0.64	0.72	111
accuracy			0.80	268
macro avg	0.81	0.78	0.78	268
weighted avg	0.80	0.80	0.79	268


```
[[143 14]
 [ 40 71]]
```

Pada tahap ini melakukan pengujian pada model dengan melakukan prediksi menggunakan data testing dan menampilkan hasil metrik dalam bentuk report mengenai model

```
pip install scikeras
```

```
Requirement already satisfied: scikeras in c:\users\225-pc27\appdata\local\programs\python\python310\lib\site-packages
```

Pada tahap ini melakukan instalasi scikeras yang digunakan instalasi library SciKeras yang bertujuan untuk membungkus model Deep Learning agar bisa berkomunikasi dengan fungsi-fungsi dari library scikit-learn.

```
# Function to generate Deep ANN model
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasClassifier
from sklearn.model_selection import GridSearchCV
import time

def make_classification_ann(Optimizer_Trial='adam', Neurons_Trial=10):
    # Creating the classifier ANN model
    model = Sequential()
    model.add(Dense(units=Neurons_Trial, input_dim=9, activation='relu'))
    model.add(Dense(units=Neurons_Trial, activation='relu'))
    model.add(Dense(units=1, activation='sigmoid'))
    model.compile(optimizer=Optimizer_Trial, loss='binary_crossentropy', metrics=['accuracy'])

    return model

#####

Parameter_Trials={'batch_size':[10,20,30],
                  'epochs':[10,20],
                  'model__Optimizer_Trial':['adam', 'rmsprop'],
                  'model__Neurons_Trial': [5,10]
                  }

# Creating the classifier ANN
classifierModel=KerasClassifier(model= make_classification_ann, verbose=0)

#####

# Creating the Grid search space
# See different scoring methods by using sklearn.metrics.SCORERS.keys()
grid_search=GridSearchCV(estimator=classifierModel, param_grid=Parameter_Trials, scoring='accuracy', cv=5,n_jobs=-1)

#####

# Measuring how much time it took to find the best params

StartTime=time.time()

# Running Grid Search for different parameters
grid_search.fit(X_train,y_train, verbose=1)

EndTime=time.time()
print("##### Total Time Taken: ", round((EndTime-StartTime)/60), 'Minutes #####')

#####

# printing the best parameters
print('\n### Best hyperparameters ###')
print(grid_search.best_params_)

print('\n### Best Accuracy ###')
print(grid_search.best_score_)
```

```
Epoch 10/20
63/63 ————— 0s 694us/step - accuracy: 0.7897 - loss: 0.4535
Epoch 11/20
63/63 ————— 0s 648us/step - accuracy: 0.7994 - loss: 0.4480
Epoch 12/20
63/63 ————— 0s 670us/step - accuracy: 0.8026 - loss: 0.4434
Epoch 13/20
63/63 ————— 0s 675us/step - accuracy: 0.8122 - loss: 0.4378
Epoch 14/20
63/63 ————— 0s 695us/step - accuracy: 0.8090 - loss: 0.4335
Epoch 15/20
63/63 ————— 0s 655us/step - accuracy: 0.8106 - loss: 0.4304
Epoch 16/20
63/63 ————— 0s 667us/step - accuracy: 0.8122 - loss: 0.4281
Epoch 17/20
63/63 ————— 0s 642us/step - accuracy: 0.8154 - loss: 0.4242
Epoch 18/20
63/63 ————— 0s 683us/step - accuracy: 0.8154 - loss: 0.4217
Epoch 19/20
63/63 ————— 0s 665us/step - accuracy: 0.8154 - loss: 0.4189
Epoch 20/20
63/63 ————— 0s 733us/step - accuracy: 0.8218 - loss: 0.4174
```

```
#### Best hyperparameters ####
{'batch_size': 10, 'epochs': 20, 'model__Neurons_Trial': 10, 'model__Optimizer_Trial': 'adam'}

#### Best Accuracy ####
0.7961032258064517
```

Pada tahap ini melakukan training dengan dataset training sebanyak 120 kali training. Dimana kombinasi yang bagus yaitu batch_size 10, epochs 20 dan neurons sebanyak 10 dan optimizer menggunakan 'adam' mendapatkan akurasi yang paling tinggi di 0.796 mendekati 0.8

Tugas Analisis :

a) Analisis percobaan 1 dan 2, jelaskan perbedaannya.

Jawaban: Perbedaan pada percobaan satu dan kedua yaitu pada algoritma model yang digunakan dalam training data iris untuk melakukan klasifikasi.

b) Kapan baiknya menggunakan MLPClassifier (scikit-learn) dan kapan menggunakan Sequential (keras) ?

Jawaban: Kita menggunakan MLPClassifier pada saat tahap awal penelitian atau jika datanya relatif sederhana seperti pada dataset iris sedangkan Sequential digunakan ketika MLPClassifier sudah tidak mampu memberikan akurasi yang diinginkan atau digunakan pada saat ketemu masalah yang kompleks.

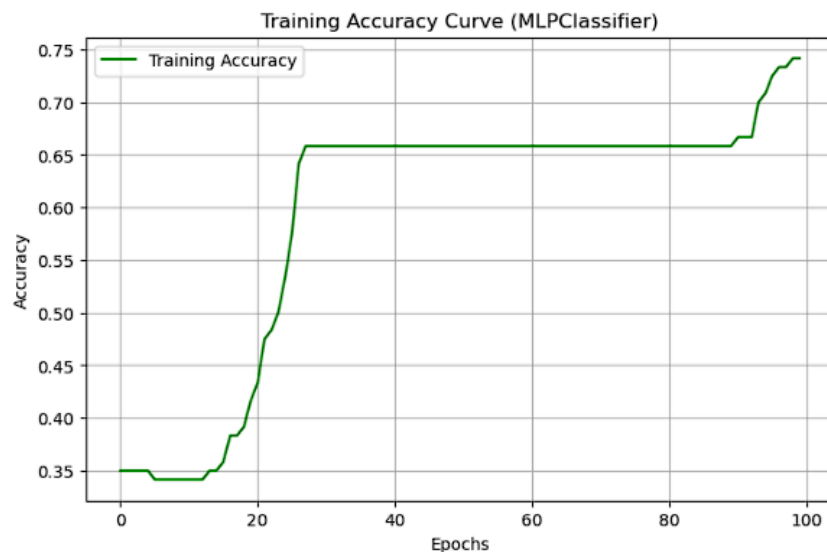
c) Coba tampilkan visualisasi hasil training akurasi dengan menggunakan MLPClassifier. Apakah bisa?

```
# Inisialisasi ulang model
mlp_manual = MLPClassifier(hidden_layer_sizes=(10,10), max_iter=1, warm_start=True)

# List untuk menampung akurasi
train_acc = []
epochs = 100
classes = np.unique(y_train)

for i in range(epochs):
    mlp_manual.partial_fit(X_train, y_train, classes=classes)
    y_train_pred = mlp_manual.predict(X_train)
    train_acc.append(accuracy_score(y_train, y_train_pred))

# Visualisasi
plt.figure(figsize=(8,5))
plt.plot(train_acc, label='Training Accuracy', color='green')
plt.title('Training Accuracy Curve (MLPClassifier)')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()
```



d) Mengapa dua tahapan ini perlu dilakukan (pada tahap 1)? Apa yang terjadi jika kita tidak membuat tahapan konversi dan pemisahan fitur dan label ini? Berikan penjelasan sesuai dengan pemahaman anda.

```
1 # Mengonversi label kategorikal menjadi numerik
2 data['species'] = data['species'].astype('category').cat.codes
```

```
1 # Memisahkan fitur dan label
2 X = data.drop('species', axis=1)
3 y = data['species']
```

Jawaban:

Perlu dilakukan karena komputer tidak memahami bahasa manusia sehingga perlu di konversi menjadi numerik dan pemisahan fitur dan label diwajibkan diawal agar saat proses latihan kita bisa memisahkan hasil dari label atau yang ingin diprediksi sehingga model dapat menebak data yang baru.

e) Untuk soal no 3, silahkan tampilkan akurasinya, serta visualisasi loss dan akurasi.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# 1. Inisialisasi ulang model MLP
# Kita pakai warm_start=True agar model tidak reset setiap kali partial_fit dipanggil
mlp_manual = MLPClassifier(
    hidden_layer_sizes=(10, 10),
    max_iter=1,
    warm_start=True,
    random_state=42
)

# 2. Siapkan list untuk menampung history
train_acc = []
train_loss = []
epochs = 100
classes = np.unique(y_train)

# 3. Proses Training Manual per Epoch
for i in range(epochs):
    # Melatih model hanya untuk 1 iterasi
    mlp_manual.partial_fit(X_train, y_train, classes=classes)

    # Catat Loss
    train_loss.append(mlp_manual.loss_)

    # Catat Akurasi Training
    y_train_pred = mlp_manual.predict(X_train)
    train_acc.append(accuracy_score(y_train, y_train_pred))
```

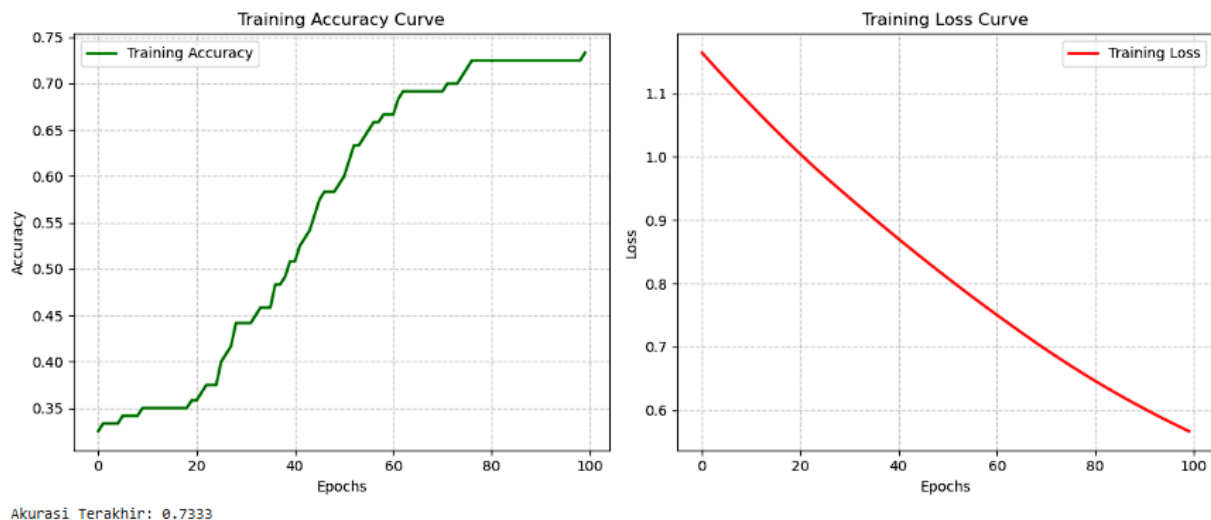


```
# --- Grafik Akurasi ---
plt.subplot(1, 2, 1)
plt.plot(train_acc, label='Training Accuracy', color='green', linewidth=2)
plt.title('Training Accuracy Curve')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend()

# --- Grafik Loss ---
plt.subplot(1, 2, 2)
plt.plot(train_loss, label='Training Loss', color='red', linewidth=2)
plt.title('Training Loss Curve')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend()

plt.tight_layout()
plt.show()

# 5. Print Akurasi Akhir
print(f"Akurasi Terakhir: {train_acc[-1]:.4f}")
```



f) Lakukan hal yang sama untuk melakukan klasifikasi apakah pasien memiliki diabetes berdasarkan beberapa fitur kesehatan seperti usia, BMI, dan lain-lain dengan menggunakan dataset Diabetes. Silahkan menggunakan MLPClassifier atau Sequential. Sertakan alasan pemilihan penggunaan librari tersebut.

Jawaban: Alasan pemilihan MLPClassifier karena diabetes ini masih dalam masalah yang gampang dan akurasiya masih tercapai menggunakan MLPClassifier yang lebih simple dalam kodingannya.

```
[1]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report, accuracy_score

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
```

```
[2]: col_names = [
    'Pregnancies',
    'Glucose',
    'BloodPressure',
    'SkinThickness',
    'Insulin',
    'BMI',
    'DiabetesPedigreeFunction',
    'Age',
    'Outcome'
]
data = pd.read_csv(url, header=None, names=col_names)
```

```
[3]: print(data.head())
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI \
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

```
[4]: print(data.info())
```

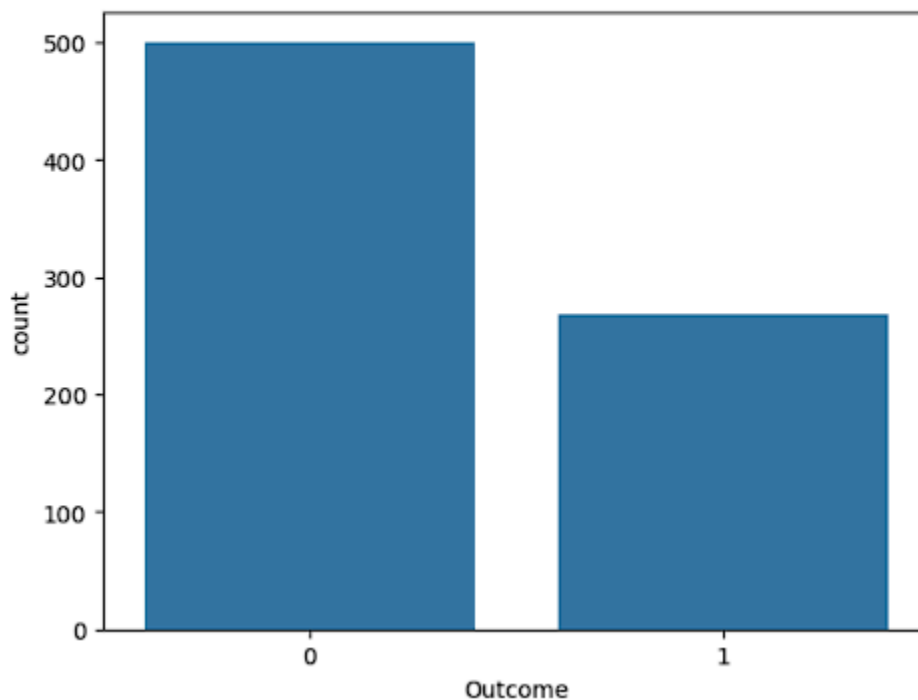
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Pregnancies                          768 non-null    int64
1   Glucose                              768 non-null    int64
2   BloodPressure                        768 non-null    int64
3   SkinThickness                        768 non-null    int64
4   Insulin                              768 non-null    int64
5   BMI                                  768 non-null    float64
6   DiabetesPedigreeFunction             768 non-null    float64
7   Age                                  768 non-null    int64
8   Outcome                              768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
```

```
[5]: data.isna().sum()
```

```
[5]: Pregnancies      0
      Glucose          0
      BloodPressure    0
      SkinThickness    0
      Insulin          0
      BMI              0
      DiabetesPedigreeFunction  0
      Age              0
      Outcome          0
      dtype: int64
```

```
[6]: import seaborn as sns
      import matplotlib.pyplot as plt

      sns.countplot(x='Outcome',data=data)
      plt.show()
```



```
[7]: data['Outcome'] = data['Outcome'].astype('category').cat.codes
```

```
[8]: x = data.drop('Outcome', axis=1)
      y = data['Outcome']
```

```
[9]: x_train, x_test, y_train, y_test = train_test_split(x,y, test_size=0.2, random_state=42)
```

```
[10]: scaler = StandardScaler()
       x_train = scaler.fit_transform(x_train)
       x_test = scaler.transform(x_test)
```

```
[11]: mlp = MLPClassifier(
        hidden_layer_sizes=(10,10),
        activation='relu',
        solver='adam',
        max_iter=1000)
```

```
[12]: #Melatih model
        history = mlp.fit(X_train, y_train)
```

```
[13]: y_pred = mlp.predict(X_test)
```

```
[14]: accuracy = accuracy_score(y_test, y_pred)
        print(f'Akurasi: {accuracy}')
```

Akurasi: 0.7662337662337663

```
[15]: print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.81	0.84	0.82	99
1	0.69	0.64	0.66	55
accuracy			0.77	154
macro avg	0.75	0.74	0.74	154
weighted avg	0.76	0.77	0.76	154

```
[16]: import seaborn as sns
        import matplotlib.pyplot as plt

        plt.figure(figsize=(8,6))
        plt.plot(mlp.loss_curve_)
        plt.title('Learning Curve')
        plt.xlabel('Epochs')
        plt.ylabel('Loss')
        plt.show()
```

